Delivery Guarantees

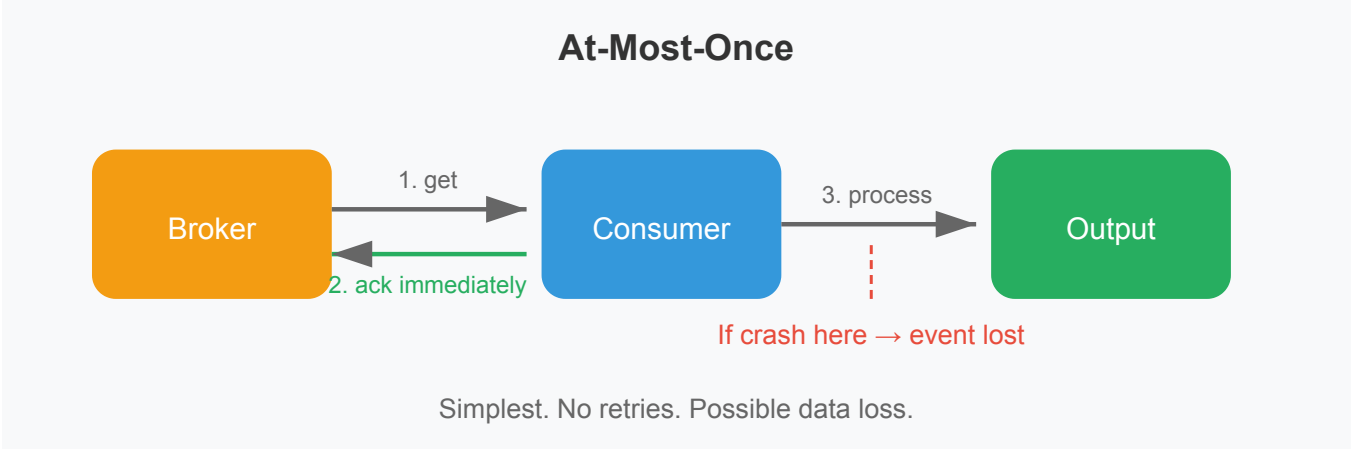
systemdesignschool.io/fundamentals/delivery-guarantees

The previous articles covered how stream processing handles unbounded data, windows, and event time. But there's a critical question we haven't addressed: when processing fails partway through, what happens to in-flight events?

A consumer reads an event, starts processing, then crashes. Is the event lost? Will it be reprocessed? Stream systems offer different guarantees about loss and duplication.

At-Most-Once: Fire and Forget

Guarantee: Each event is processed zero or one time. Events may be lost on failure.



How it works: The consumer acknowledges receipt (commits its offset) immediately, before processing completes. If processing then crashes, the event is gone—the broker already marked it consumed and won't redeliver.

Example scenario: A logging pipeline processes events to count errors. The consumer reads event X, commits its offset, then crashes while writing to the database. When it restarts, it resumes from after event X. Event X is lost.

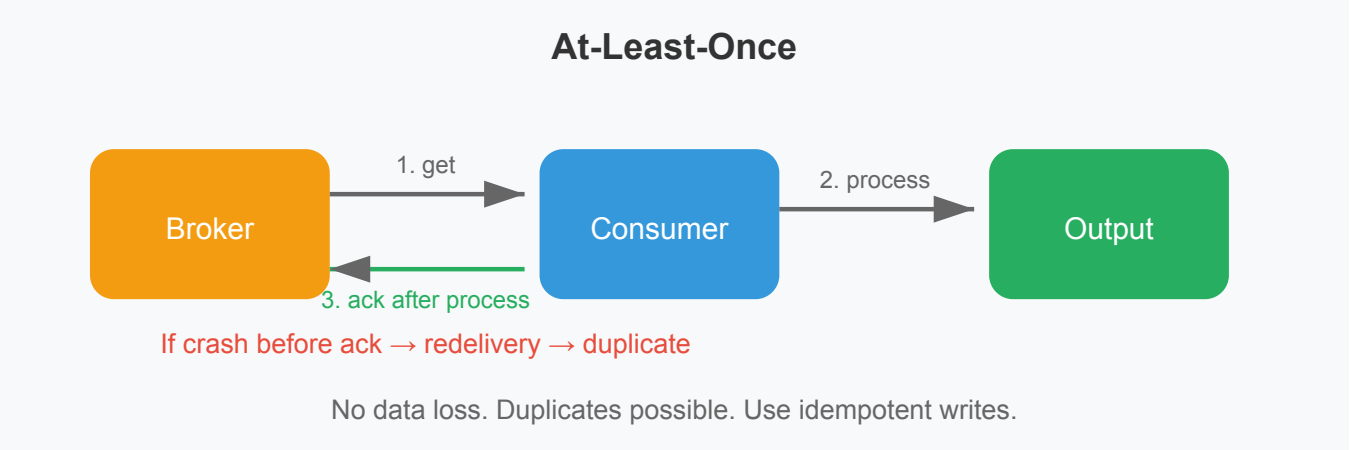
Use when: Occasional data loss is acceptable and speed matters most. Metrics pipelines where a few missing data points don't affect overall trends. Log aggregation where some gaps are tolerable.

Trade-offs:

- Simplest implementation
- Lowest latency (no retry overhead)
- Data loss on failures

At-Least-Once: Never Lose, May Duplicate

Guarantee: Each event is processed one or more times. No events are lost, but duplicates are possible.



How it works: The consumer processes the event fully, then acknowledges. If it crashes before acknowledging, the broker redelivers the event when the consumer restarts. The consumer processes it again—a duplicate.

Example scenario: Same logging pipeline. The consumer reads event X, writes to the database, then crashes before committing its offset. When it restarts, the broker redelivers event X. The database now has two copies of the same error count.

Making it work: Downstream systems must handle duplicates. Strategies include:

- **Idempotent operations:** An upsert (INSERT or UPDATE by key) produces the same result whether executed once or twice. A plain INSERT creates duplicates.
- **Deduplication by event ID:** Store a unique event ID with each record. Before processing, check if you've seen this ID before.
- **Natural idempotency:** "Set balance to $100" is idempotent. "Add $10 to balance" is not.

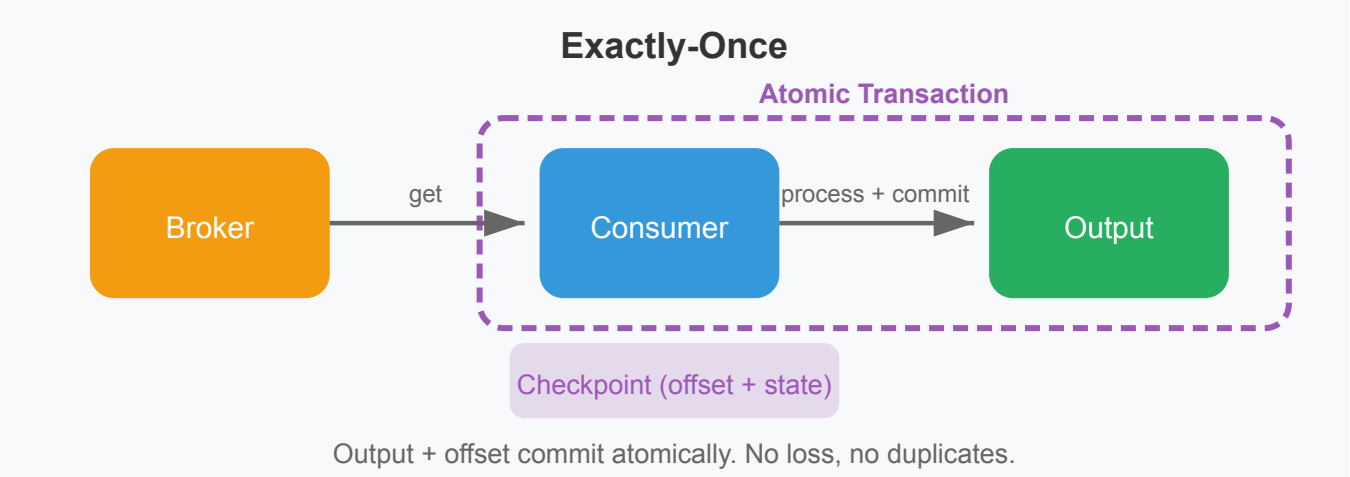
Use when: Data loss is unacceptable and you can design downstream operations to be idempotent. Most production stream processing uses this approach.

Trade-offs:

- No data loss
- Must handle duplicates in downstream logic
- Slightly higher latency than at-most-once

Exactly-Once: Neither Loss Nor Duplication

Guarantee: Each event is processed exactly once, even across failures. No loss, no duplicates.



How it works: The system bundles the processing result and the input acknowledgment into a single atomic transaction. Either both commit (result is written AND event is marked consumed) or neither does. On failure, the system rolls back to a consistent state.

Modern stream processors like Flink and Kafka Streams achieve this through **distributed snapshots** (checkpoints). The framework periodically captures the entire pipeline state: what's been consumed, what's in-flight, what's been output. On failure, restore to the last checkpoint and resume. Events between the checkpoint and the failure get reprocessed, but the outputs are either suppressed (if already committed) or written exactly once.

The catch: "Exactly-once" typically applies *within the stream processing system*. If your pipeline writes to external systems (a database, an API), achieving exactly-once end-to-end requires either:

- The sink supporting transactions that coordinate with the checkpoint, or
- Idempotent writes to the sink, or
- The ability to roll back external writes on failure

Without this, you might have exactly-once processing but at-least-once delivery to the sink.

Use when: Correctness is critical—financial transactions, billing calculations, inventory updates where duplicates or gaps cause real problems.

Trade-offs:

- Strongest guarantee
- Higher latency (checkpointing overhead)
- More complex implementation
- Requires coordination with external systems for true end-to-end guarantees

Practical Guidance

Guarantee	Data Loss	Duplicates	Complexity	Latency
At-most-once	Possible	No	Low	Lowest
At-least-once	No	Possible	Medium	Medium
Exactly-once	No	No	High	Highest

Many production systems use **at-least-once with idempotent operations**. This achieves "effectively-once" semantics with lower complexity than true exactly-once checkpointing:

- Use upserts instead of inserts
- Include unique event IDs and deduplicate at the sink
- Design operations to be naturally idempotent where possible

This approach works well because idempotency pushes the complexity to application logic (which you control) rather than distributed transaction protocols (which are hard to get right).

What's Next

We've covered the fundamentals of batch and stream processing: the models, windowing, time semantics, and delivery guarantees. The next section explores **hybrid architectures**—Lambda and Kappa patterns that combine batch and stream processing for systems that need both historical completeness and real-time updates.